

Week-7 Practical

1. Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

```
GNU nano 8.7.1      src/memory_addresses.c *
#include <stdio.h>
#include <stdlib.h>

int global_var = 10;

static int static_var = 20;

void code_function() {
    printf("Code address   : %p\n", (void *)code_function);
}

int main() {
    int stack_var = 30;
    int *heap_var = malloc(sizeof(int));

    *heap_var = 40;

    printf("Memory Addresses\n");
    printf("-----\n");

    code_function();

    printf("Global address : %p\n", (void *)&global_var);
    printf("Static address : %p\n", (void *)&static_var);
    printf("Heap address   : %p\n", (void *)heap_var);
    printf("Stack address  : %p\n", (void *)&stack_var);

    free(heap_var);

    return 0;
}
```

Output:

```
akhil@Akhil-laptop: ~$ cd ~/OS_Skill/Week-7
akhil@Akhil-laptop:~/OS_Skill/Week-7$ nano src/memory_addresses
akhil@Akhil-laptop:~/OS_Skill/Week-7$ gcc src/memory_addresses.
c -o bin/memory_addresses
akhil@Akhil-laptop:~/OS_Skill/Week-7$ ./bin/memory_addresses
Memory Addresses
-----
Code address   : 0x5f8dd46911c9
Global address : 0x5f8dd4694010
Static address : 0x5f8dd4694014
Heap address   : 0x5f8de5084010
Stack address  : 0x7ffc011a4c7c
```

2. Make use of the /proc//maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

```
GNU nano 8.7.1 src/memory_maps.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 100;

int main() {
    int stack_var = 200;

    int *heap_var = malloc(sizeof(int));
    *heap_var = 300;

    printf("Process PID: %d\n", getpid());
    printf("Global address: %p\n", (void *)&global_var);
    printf("Stack address : %p\n", (void *)&stack_var);
    printf("Heap address : %p\n", (void *)heap_var);

    printf("\nProcess is running...\n");
    printf("Use another terminal to inspect /proc/%d/maps\n", getpid());

    while (1) {
        sleep(1);
    }

    free(heap_var);

    return 0;
}
```

Output:

```
akhil@Akhil-laptop:~/OS_SKILL/Week-4$
c
akhil@Akhil-laptop:~/OS_SKILL/Week-4$ gcc src/memory_addresses
c -o memory_addresses
akhil@Akhil-laptop:~/OS_SKILL/Week-4$ gcc src/memory_addresses
c -o memory_addresses
akhil@Akhil-laptop:~/OS_SKILL/Week-4$ ./bin/memory_addresses
Memory Addresses
Code address : 0x401000
Static address: 0x404000
Heap address : 0x602000
Stack address : 0x7ffffde000
akhil@Akhil-laptop:~/OS_SKILL/Week-4$ nano src/memory_map.c
akhil@Akhil-laptop:~/OS_SKILL/Week-4$ gcc src/memory_map.c -o
bin/memory_map
akhil@Akhil-laptop:~/OS_SKILL/Week-4$ ./bin/memory_map
Process PID: 729
Code address : 0x402000-0x402910
Stack address : 0x7fffffde000
Heap address : 0x602000-0x602910

Process is running...
Use another terminal to inspect /proc/729/maps

akhil@Akhil-laptop:~/OS_SKILL/Week-4$ grep heap /proc/729/
729
akhil@Akhil-laptop:~/OS_SKILL/Week-4$ grep stack /proc/729/
729
akhil@Akhil-laptop:~/OS_SKILL/Week-4$ psmap 729
729: memory map:
00400000-00452000 r-xp 00000000 08:00 0
[heap]
60600000-60622000 rw-p 00000000 08:00 0
60622000-60623000 rw-p 00000000 08:00 0
60623000-60624000 r-xp 00000000 08:00 0
60624000-60627000 rw-p 00000000 08:00 0
7ffffde1cab000-7ffffde1cc000 rw-p 00000000 00:00 0
7ffffde1cc000-7ffffde1e000 rw-p 00000000 00:00 0
7ffffde1e000-7ffffde1f000 r-xp 00000000 00:00 0
7ffffde1f000-7ffffde20000 ---p 00000000 00:00 0
7ffffde20000-7ffffde22000 rw-p 00000000 00:00 0
7ffffde22000-7ffffde23000 r-xp 00000000 00:00 0
7ffffde23000-7ffffde24000 r-p 00000000 00:00 0
7ffffde24000-7ffffde25000 rw-p 00000000 00:00 0
7ffffde25000-7ffffde27000 rw-p 00000000 00:00 0
7ffffde27000-7ffffde29000 r-xp 00000000 00:00 0
7ffffde29000-7ffffde2b000 rw-p 00000000 00:00 0
7ffffde2b000-7ffffde2d000 rw-p 00000000 00:00 0
7ffffde2d000-7ffffde2f000 r-xp 00000000 00:00 0
7ffffde30000-7ffffde38000 r-p 00000000 00:00 0
7ffffde39000-7ffffde35000 r-p 00000000 00:00 0
7ffffde37000-7ffffde39000 rw-p 00000000 00:00 0
7ffffde3f000-7ffffde39000 r-p 00000000 00:00 0
7ffffde3f000-7ffffde41000 r-xp 00000000 00:00 0
7ffffde3f000-7ffffde41000 [vvar]
7ffffde31000-7ffffde44000 [vdso]
7ffffde44000-7ffffde45000 [vvar]
```